

System Architecture & Technology Stack

AI Trust & Safety Auditing Agent for AI-Based AIS

Project: Project 1 of 3
Client: The Digital Forge (Fortino Global Education)
Author: Wenguang Li
Date: March 24, 2026

Table of Contents

- Part 1: System Architecture (5-7 Pages).....2
 - 1. High-Level Architecture Overview2
 - 1.1 Architectural Pattern: Council of Experts.....2
 - 1.2 System Architecture Diagram (Conceptual).....3
 - 2. Module Boundaries and Responsibilities4
 - 2.1 Module 1: Governance & Compliance Expert.....4
 - 2.2 Module 2: Fairness & Bias Expert4
 - 2.3 Module 3: Security & Privacy Expert.....5
 - 2.4 Module 4: Explainability & Audit Trail Expert.....5
 - 2.5 Module 5: Accuracy & Performance Expert.....5
 - 3. Data Flow Architecture.....6
 - 3.1 Phase 1: Input Processing6
 - 3.2 Phase 2: Independent Module Evaluation6
 - 3.3 Phase 3: Synthesis & Interdependency Resolution7
 - 3.4 Phase 4: Report Generation.....8
 - 4. Interface Contracts (I/O Specifications)9
 - 4.1 Module Interface Contract (All 5 Modules).....9
 - 4.2 Synthesis Layer Interface Contract 10
 - 4.3 Report Generator Interface Contract..... 12
 - 5. Deployment Architecture (NYU AI Sandbox) 12
 - 5.1 Component Deployment..... 12
 - 5.2 Execution Flow 13
- Part 2: Technology Stack Summary (1-2 Pages)..... 14
 - 6. Selected Technology Stack 14
 - 6.1 Core Technologies 14

6.2 Supporting Libraries	14
6.3 Architecture Choices	14
6.4 NYU AI Sandbox Constraints & Compliance	16
6.5 Deployment Checklist for NYU Sandbox	16
6.6 Technology Roadmap	17
7. Interface Design Principles for Projects 2 & 3	17
7.1 Modularity Guidelines	17
7.2 Testing Strategy	18
8. Risks & Mitigation.....	18
9. Success Metrics (Architecture Quality).....	19
10. Conclusion.....	19

Executive Summary

This document defines the system architecture, module boundaries, data flow, and interface contracts for the AI Trust & Safety Auditing Agent. The architecture follows a “Council of Experts” pattern where five specialized modules independently evaluate AI-based Accounting Information Systems (AIS) across distinct dimensions, then synthesize results through a three-stage peer review process. The selected technology stack—centered on Python and ChatGPT API—is designed for deployment in the NYU AI Sandbox environment and supports iterative development by Projects 2 and 3.

Part 1: System Architecture

1. High-Level Architecture Overview

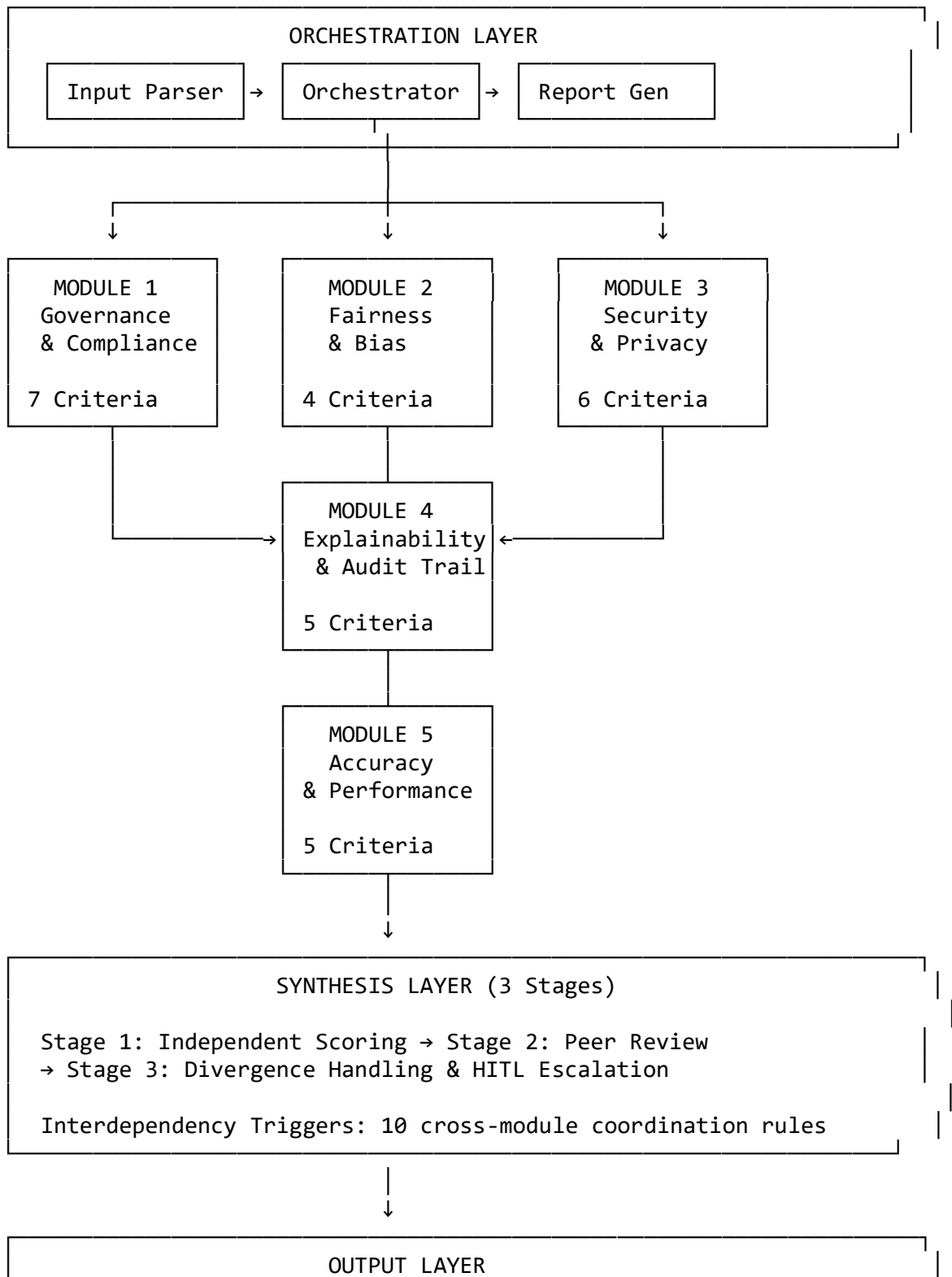
1.1 Architectural Pattern: Council of Experts

The system implements an **ensemble-based audit architecture** where multiple specialized modules (“experts”) independently evaluate an AI system, then collaborate to produce a comprehensive audit report. This approach mirrors real-world audit practices where specialized teams assess different dimensions (financial, compliance, IT) before synthesizing findings.

Core Principles: 1. **Modularity:** Each expert module operates independently with clear boundaries 2. **Specialization:** Modules focus on distinct audit dimensions (Governance, Fairness, Security, Explainability, Accuracy) 3. **Peer Review:** Three-stage synthesis process validates findings across modules 4. **Transparency:** All scoring, triggers, and escalations

are auditable 5. **Human-in-the-Loop (HITL):** Divergent findings automatically escalate to human review

1.2 System Architecture Diagram (Conceptual)



- Audit Scorecard (28 criterion scores + module aggregates)
- Material Weakness Report (SOX 404 classification)
- Evidence Traceability Log
- Interdependency Trigger Log
- HITL Escalation Flags (if divergence ≥ 20 points)

2. Module Boundaries and Responsibilities

2.1 Module 1: Governance & Compliance Expert

Scope: AI governance structures, policies, compliance documentation, and organizational oversight.

Criteria (7): - G1.1: AI Governance Policy - G1.2: Risk Management & Oversight - G1.3: SOX 404 Internal Controls - G1.4: AI System Inventory - G1.5: Training & Competency - G1.6: Records of Processing Activities (ROPA) - GDPR Art. 30 - G1.7: Data Protection Impact Assessment (DPIA) Governance - GDPR Art. 35

Boundaries: - **IN SCOPE:** Governance documentation, policy existence, oversight structures, training programs, GDPR compliance documentation - **OUT OF SCOPE:** Technical implementation of fairness/security measures (covered by other modules)

Dependencies: - Module 2 (Fairness): G1.1 governance gaps trigger fairness escalation - Module 3 (Security): G1.6 ROPA completeness affects privacy assessment - Module 4 (Explainability): G1.2 risk oversight impacts transparency requirements

2.2 Module 2: Fairness & Bias Expert

Scope: Algorithmic fairness, bias detection/mitigation, special category data handling, automated decision-making compliance.

Criteria (4): - F2.1: Fairness Metrics with Special Category Data Compliance - GDPR Art. 9 - F2.2: Bias Monitoring - F2.3: Bias Mitigation - F2.4: Automated Decision-Making Opt-Out - GDPR Art. 22, CCPA §1798.185

Boundaries: - **IN SCOPE:** Fairness testing results, bias metrics, protected attribute analysis, opt-out mechanisms - **OUT OF SCOPE:** General data accuracy (Module 5), security of fairness testing data (Module 3)

Dependencies: - Module 1 (Governance): G1.1 governance weakness triggers fairness testing escalation - Module 4 (Explainability): Fairness failures require enhanced explainability - Module 5 (Accuracy): Bias often correlates with subgroup accuracy disparities

2.3 Module 3: Security & Privacy Expert

Scope: Cybersecurity, adversarial robustness, data protection, breach detection/notification, privacy compliance.

Criteria (6): - S3.1: Encryption (data at rest, in transit) - S3.2: Adversarial Robustness - S3.3: Access Control - S3.4: Privacy by Design - GDPR Art. 25 - S3.5: Breach Detection & Notification - GDPR Art. 33-34, CCPA §1798.150 - S3.6: Data Minimization & Retention - GDPR Art. 5, CCPA §1798.105

Boundaries: - **IN SCOPE:** Security controls, privacy safeguards, breach response procedures, data lifecycle management - **OUT OF SCOPE:** Business continuity planning (unless related to data protection), non-AI security

Dependencies: - Module 1 (Governance): G1.6 ROPA informs S3.6 data retention assessment - Module 4 (Explainability): S3.3 access controls affect audit trail integrity - Module 5 (Accuracy): S3.2 adversarial robustness impacts model accuracy

2.4 Module 4: Explainability & Audit Trail Expert

Scope: AI decision transparency, audit trail completeness, model documentation, human review mechanisms, consumer rights (CCPA).

Criteria (5): - E4.1: AI Decision Explainability with GDPR Transparency - GDPR Art. 13-14 - E4.2: Audit Trail Completeness - E4.3: Model Documentation - E4.4: Human Review Mechanism - GDPR Art. 22 - E4.5: CCPA Opt-Out & Right to Know - CCPA §1798.100, §1798.185

Boundaries: - **IN SCOPE:** Explainability methods (SHAP, LIME), audit logs, documentation quality, consumer notice mechanisms - **OUT OF SCOPE:** Model accuracy validation (Module 5), fairness of explanations (Module 2)

Dependencies: - ALL MODULES: Explainability failure ($E4.1 \leq 0.5$) flags all findings as “Lower Confidence” - Module 1 (Governance): E4.3 documentation quality reflects G1.2 oversight - Module 2 (Fairness): F2.4 opt-out implementation requires E4.5 consumer notice

2.5 Module 5: Accuracy & Performance Expert

Scope: Model validation, data accuracy, drift monitoring, performance metrics, data subject correction rights.

Criteria (5): - A5.1: Model Validation & Data Accuracy - GDPR Art. 5(1)(d) - A5.2: Drift Monitoring with GDPR Compliance - A5.3: Performance Metrics Materiality Linkage - A5.4: Personal Data Inference Accuracy - GDPR Art. 5(1)(d) - A5.5: Data Subject Correction Rights - GDPR Art. 16, CCPA §1798.106

Boundaries: - **IN SCOPE:** Model performance validation, accuracy metrics, drift detection, data quality, correction mechanisms - **OUT OF SCOPE:** Root cause analysis of accuracy issues (may involve fairness/security problems)

Dependencies: - Module 2 (Fairness): A5.1 accuracy failures may reveal bias (trigger F2.1 re-evaluation) - Module 3 (Security): A5.2 drift may indicate adversarial attacks (trigger S3.2 review) - Module 4 (Explainability): A5.3 performance metrics must be explainable

3. Data Flow Architecture

3.1 Phase 1: Input Processing

Inputs (Provided by Auditor): 1. **AI System Metadata:** - System name, version, deployment date - Business process supported (e.g., credit scoring, fraud detection) - Geographic scope (determines GDPR/CCPA applicability)

2. **Evidence Artifacts (28 sets, one per criterion):**
 - Documents: Policies, procedures, test reports, logs
 - Data files: Fairness metrics, audit trails, performance data
 - Screenshots: UI elements, dashboards, configuration settings
3. **Context Information:**
 - Materiality threshold (financial)
 - GDPR/CCPA jurisdiction flags
 - Special category data involvement (Yes/No)

Processing: - Input Parser validates structure (JSON schema validation) - Evidence is mapped to criterion IDs (G1.1 - A5.5) - Geographic exemptions applied (N/A for non-applicable criteria)

3.2 Phase 2: Independent Module Evaluation

Flow for Each Module:

Evidence Artifacts (filtered to module criteria)

↓

[Criterion-Level Evaluation Loop]

For each criterion (e.g., G1.1):

1. ChatGPT analyzes evidence against rubric
2. Assigns score 0.0-1.0
3. Provides justification
4. Classifies: Pass (>0.75), Sig Def ($0.51-0.75$), Mat Weak (≤ 0.5)

↓

[Module-Level Aggregation]

Weighted average of criterion scores → Module Score
(Criteria with N/A excluded from weight calculation)

↓

[Module Output]

```
{  
  "module_id": "M1_Governance",  
  "module_score": 0.72,  
  "criterion_scores": [  

```

```

    {"criterion_id": "G1.1", "score": 0.8, "classification": "Pass"},
    {"criterion_id": "G1.2", "score": 0.6, "classification": "Sig Def"},
    ...
  ],
  "justifications": [...],
  "evidence_references": [...]
}

```

Key Characteristics: - Modules run **independently** (no inter-module communication at this stage) - Scoring is **deterministic** given the same evidence (reproducibility) - Each module produces **self-contained output** (no side effects)

3.3 Phase 3: Synthesis & Interdependency Resolution

Stage 1: Independent Scoring (Complete) All 5 modules have produced outputs.

Stage 2: Peer Review & Interdependency Triggers

The Synthesis Layer applies **10 interdependency triggers**:

1. **Governance → Fairness Escalation**
 - IF G1.1 (Governance Policy) ≤ 0.5 (Material Weakness)
 - THEN escalate F2.1 (Fairness Metrics) to “Requires Enhanced Testing”
2. **Governance → Security Escalation**
 - IF G1.2 (Risk Oversight) ≤ 0.5
 - THEN escalate S3.1, S3.3 (Encryption, Access Control) review
3. **ROPA Completeness → Privacy Assessment**
 - IF G1.6 (ROPA) is incomplete (<0.75)
 - THEN flag S3.4, S3.6 (Privacy by Design, Data Retention) as “Higher Risk”
4. **Explainability Failure → Global Confidence Flag**
 - IF E4.1 (Explainability) ≤ 0.5
 - THEN mark ALL findings as “Lower Confidence” (insufficient transparency to trust audit)
5. **Fairness Failure → Accuracy Re-evaluation**
 - IF F2.1 (Fairness Metrics) ≤ 0.5
 - THEN trigger A5.4 (Inference Accuracy) subgroup analysis
6. **Drift Detection → Adversarial Review**
 - IF A5.2 (Drift Monitoring) detects significant drift
 - THEN trigger S3.2 (Adversarial Robustness) investigation
7. **Breach Notification Failure → All Privacy Criteria**
 - IF S3.5 (Breach Notification) ≤ 0.5
 - THEN escalate all GDPR/CCPA criteria (G1.6, G1.7, F2.4, S3.4, S3.6, E4.5, A5.5)
8. **Audit Trail Gaps → Evidence Reliability**
 - IF E4.2 (Audit Trail) ≤ 0.5
 - THEN flag all evidence-based scores as “Lower Reliability”

9. Automated Decision Opt-Out Missing → Transparency

- IF F2.4 (Opt-Out) not implemented
- THEN escalate E4.5 (CCPA Right to Know) review

10. Materiality Calculation Adjustment

- IF S3.5 (Breach) or multiple GDPR/CCPA criteria fail
- THEN add potential fines (€20M/4% turnover, CCPA \$100-750/consumer) to materiality

Data Flow:

Module Outputs (5) → Interdependency Trigger Engine

↓

Trigger Activation Log (which triggers fired)

↓

Updated Findings with Flags:

- "Enhanced Testing Required"
- "Higher Risk"
- "Lower Confidence"
- "Lower Reliability"

↓

Stage 3: Divergence Handling

Stage 3: Divergence Handling & HITL Escalation

Divergence Check: For each criterion evaluated by multiple modules (cross-checks), calculate score divergence:

$\text{Divergence} = |\text{Score_ModuleA} - \text{Score_ModuleB}|$

HITL Escalation Rule:

IF Divergence ≥ 0.20 (20 percentage points)

THEN escalate to Human-in-the-Loop review

- Present both module outputs
- Show evidence used by each module
- Request human adjudication
- Log final decision

Example: - Module 1 scores G1.2 (Risk Oversight) = 0.8 (Pass) - Module 3 cross-checks G1.2 from security perspective = 0.55 (Sig Def) - Divergence = $|0.8 - 0.55| = 0.25 \geq 0.20 \rightarrow$

HITL ESCALATION

3.4 Phase 4: Report Generation

Inputs: - Module outputs (5) - Interdependency trigger log - HITL adjudications (if any) - Materiality calculations

Outputs:

1. Audit Scorecard:

- 28 criterion scores (0.0-1.0 scale)
 - 5 module aggregate scores
 - Overall system score (weighted average)
 - SOX 404 classifications (Pass, Sig Def, Mat Weak)
2. **Material Weakness Report:**
 - List of Material Weaknesses (score ≤ 0.5)
 - Materiality calculations (financial impact)
 - Regulatory citations (GDPR articles, CCPA sections)
 - Remediation recommendations
 3. **Evidence Traceability Log:**
 - For each score, references to evidence artifacts used
 - Enables audit trail reconstruction
 4. **Interdependency Trigger Log:**
 - Which triggers activated
 - Resulting escalations/flags
 - Demonstrates comprehensive cross-module analysis
 5. **HITL Escalation Summary:**
 - Divergent findings requiring human review
 - Human adjudication results
 - Ensures transparency in human judgment
-

4. Interface Contracts (I/O Specifications)

4.1 Module Interface Contract (All 5 Modules)

Input Schema:

```
{
  "module_input": {
    "ai_system_metadata": {
      "system_name": "string",
      "system_version": "string",
      "deployment_date": "ISO-8601 date",
      "business_process": "string",
      "geographic_scope": ["US", "EU", "Other"],
      "special_category_data": "boolean"
    },
    "criteria": [
      {
        "criterion_id": "string (e.g., G1.1)",
        "evidence_artifacts": [
          {
            "artifact_type": "document|data|screenshot",
            "artifact_name": "string",
            "artifact_content": "string|base64",
```

```

        "upload_timestamp": "ISO-8601 datetime"
    }
],
    "exemption_status": "applicable|not_applicable|pending"
}
],
    "materiality_threshold": "number (USD)",
    "audit_context": {
        "audit_date": "ISO-8601 date",
        "auditor_name": "string",
        "audit_purpose": "string"
    }
}
}
}

```

Output Schema:

```

{
    "module_output": {
        "module_id": "string (M1_Governance|M2_Fairness|M3_Security|M4_Explainability|M5_Accuracy)",
        "module_name": "string",
        "module_score": "number (0.0-1.0)",
        "execution_timestamp": "ISO-8601 datetime",
        "criterion_results": [
            {
                "criterion_id": "string",
                "criterion_name": "string",
                "score": "number (0.0-1.0)",
                "classification": "Pass|Significant_Deficiency|Material_Weakness|N/A"
            },
            {
                "justification": "string (AI-generated reasoning)",
                "evidence_references": ["array of artifact names"],
                "regulatory_citations": ["array of relevant regulations"],
                "confidence_level": "High|Medium|Low"
            }
        ],
        "module_level_findings": "string (summary)",
        "recommended_actions": ["array of strings"],
        "processing_metadata": {
            "llm_model": "string (e.g., gpt-4)",
            "token_usage": "number",
            "execution_time_seconds": "number"
        }
    }
}

```

4.2 Synthesis Layer Interface Contract

Input:

```

{
  "synthesis_input": {
    "module_outputs": [
      /* Array of 5 module outputs (from 4.1 Output Schema) */
    ],
    "interdependency_triggers_config": {
      "triggers": [
        {
          "trigger_id": "string",
          "condition": "string (logical expression)",
          "action": "string (escalation/flag)",
          "affected_criteria": ["array of criterion IDs"]
        }
      ]
    },
    "hitl_escalation_threshold": "number (default 0.20)"
  }
}

```

Output:

```

{
  "synthesis_output": {
    "overall_audit_score": "number (0.0-1.0)",
    "overall_classification": "Pass|Significant_Deficiency|Material_Weakness"
  },
  "module_scores": {
    "governance": "number",
    "fairness": "number",
    "security": "number",
    "explainability": "number",
    "accuracy": "number"
  },
  "material_weaknesses": [
    {
      "criterion_id": "string",
      "criterion_name": "string",
      "score": "number",
      "materiality_impact_usd": "number",
      "regulatory_exposure": ["GDPR Art. X", "CCPA §Y"],
      "remediation_priority": "Critical|High|Medium"
    }
  ],
  "interdependency_triggers_activated": [
    {
      "trigger_id": "string",
      "trigger_description": "string",
      "source_criterion": "string",
      "affected_criteria": ["array"],
      "action_taken": "string"
    }
  ]
}

```

```

    }
  ],
  "hitl_escalations": [
    {
      "criterion_id": "string",
      "divergence_score": "number",
      "module_scores": {
        "module_a": "number",
        "module_b": "number"
      },
      "status": "pending|adjudicated",
      "human_decision": "string (if adjudicated)",
      "final_score": "number (if adjudicated)"
    }
  ],
  "audit_summary": "string (executive summary)",
  "next_steps": ["array of recommended actions"]
}

```

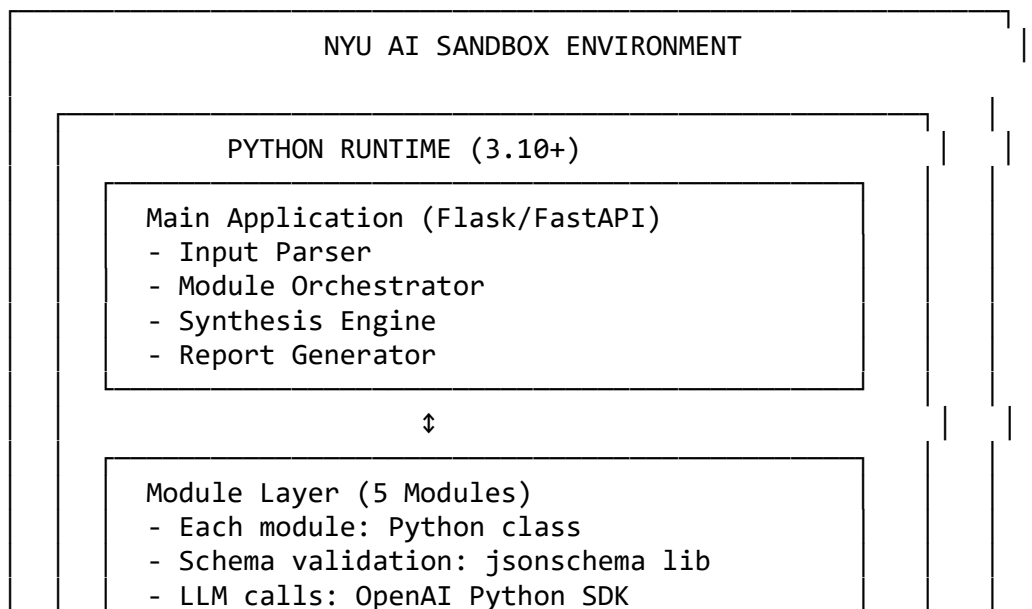
4.3 Report Generator Interface Contract

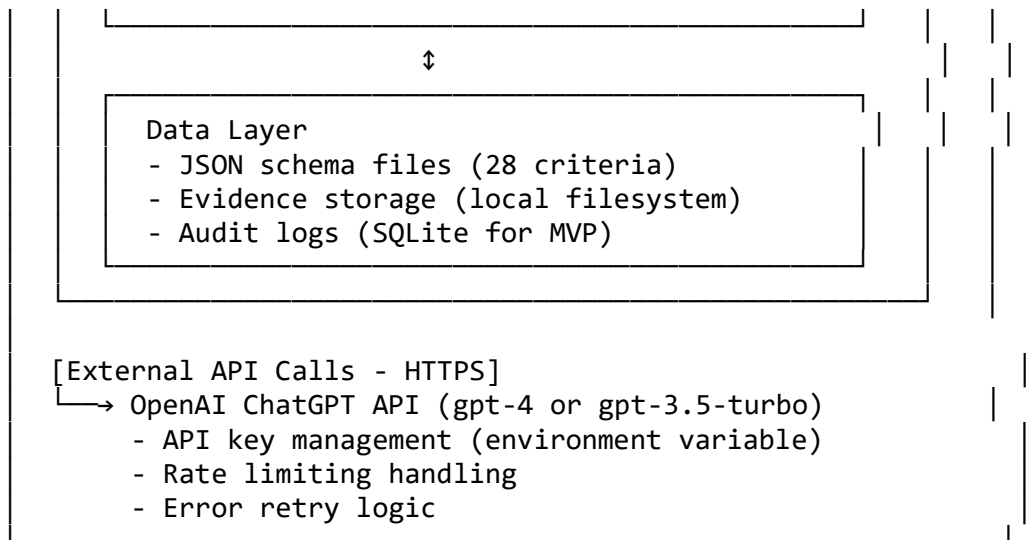
Input: Synthesis output (from 4.2) + formatting preferences

Output: - PDF report (formatted audit report) - JSON scorecard (machine-readable) - CSV evidence traceability log

5. Deployment Architecture (NYU AI Sandbox)

5.1 Component Deployment





5.2 Execution Flow

1. **Auditor uploads evidence via web interface**
 - Files stored in sandbox filesystem
 - Metadata extracted and validated
 2. **Orchestrator distributes work to modules**
 - Each module processes its criteria independently
 - ChatGPT API calls for AI-powered evaluation
 3. **Synthesis Layer combines results**
 - Interdependency triggers applied
 - Divergence detection
 - HITL escalation flags generated
 4. **Report Generator produces outputs**
 - PDF report rendered (ReportLab library)
 - JSON scorecard exported
 - Evidence traceability CSV generated
 5. **Auditor downloads outputs**
 - All outputs available via web interface
 - Audit log persisted for reproducibility
-

Part 2: Technology Stack Summary

6. Selected Technology Stack

6.1 Core Technologies

Component	Technology	Version	Justification
Primary Language	Python	3.10+	NYU Sandbox standard; rich ecosystem for AI/ML; jsonschema, OpenAI SDK support
LLM Provider	OpenAI ChatGPT	GPT-4	User-specified; state-of-the-art reasoning; reliable API; JSON mode for structured output
Web Framework	FastAPI	0.100+	Modern async framework; automatic OpenAPI docs; WebSocket support for real-time updates; lightweight for sandbox
Schema Validation	jsonschema	4.x	Python standard for JSON schema validation; ensures input/output contract compliance
PDF Generation	ReportLab	4.x	Professional PDF reports; Python-native; no external dependencies
Data Storage (MVP)	SQLite	3.x	Embedded database; zero configuration; sufficient for sandbox prototype; easy migration to PostgreSQL later
Testing	pytest	7.x	Python standard testing framework; supports async tests; extensive plugin ecosystem
API Client	OpenAI Python SDK	1.x	Official OpenAI library; handles authentication, retries, rate limiting

6.2 Supporting Libraries

Data Processing: - pandas 2.x: Evidence data manipulation, CSV export - pydantic 2.x: Data validation and settings management

Web Interface (Optional for MVP): - streamlit 1.x: Rapid prototyping of web UI (alternative to FastAPI frontend) - bootstrap 5.x: Responsive UI styling

Utilities: - python-dotenv: Environment variable management (API keys) - requests: HTTP client for external API calls - logging: Built-in Python logging for audit trails

6.3 Architecture Choices

6.3.1 Why ChatGPT (User-Specified)

Strengths: - **Structured Output:** JSON mode ensures consistent schema compliance - **Contextual Understanding:** Can analyze nuanced audit evidence (policies, test reports) - **Regulatory Knowledge:** Pre-trained on GDPR, CCPA, SOX documentation - **Chain-of-Thought:** Provides justifications for scores (transparency)

Integration Approach:

```

import openai

# Per-criterion evaluation
response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[
        {"role": "system", "content": "You are an AI audit expert evaluating criterion G1.1..."},
        {"role": "user", "content": f"Evidence: {evidence_text}"}
    ],
    response_format={"type": "json_object"}, # Enforce JSON output
    temperature=0.2 # Low temperature for consistency
)

# Parse structured output
result = json.loads(response.choices[0].message.content)
criterion_score = result["score"] # 0.0-1.0
justification = result["justification"]

```

Rate Limiting Strategy: - Batch criterion evaluations where possible - Implement exponential backoff on 429 errors - Cache results for identical evidence (within audit session)

6.3.2 Why FastAPI Over Flask

Requirement	FastAPI	Flask	Winner
Async support (parallel module execution)	Native	Requires extensions	FastAPI
Automatic API docs (OpenAPI/Swagger)	Built-in	Requires flask-swagger	FastAPI
Data validation (Pydantic)	Native	Requires marshmallow	FastAPI
WebSocket (real-time audit progress)	Built-in	Requires flask-socketio	FastAPI
Learning curve	Moderate	Easy	Flask
NYU Sandbox compatibility	Yes	Yes	Tie

Decision: FastAPI for async execution and built-in validation. Flask acceptable if team prefers simplicity.

6.3.3 Why SQLite for MVP

Sandbox Constraints: - No managed database services in NYU Sandbox - Need local, file-based storage - Prototype focus (production can migrate to PostgreSQL)

SQLite Advantages: - Zero configuration (single .db file) - ACID compliance - Sufficient performance for MVP workload (<100 audits) - Easy backup (copy .db file)

Migration Path to PostgreSQL (Project 3):

Abstraction Layer using SQLAlchemy ORM

```
from sqlalchemy import create_engine
```

SQLite (MVP)

```
engine = create_engine("sqlite:///audit_agent.db")
```

PostgreSQL (Production) - same code works

```
engine = create_engine("postgresql://user:pass@host/dbname")
```

6.4 NYU AI Sandbox Constraints & Compliance

Constraint	Requirement	Our Solution
No Internet Access (Restricted Mode)	May block external API calls	Requires sandbox "API Access" mode for OpenAI calls; document in setup guide
Python Version	3.8+ typically available	Our stack requires 3.10+, verify availability
File Storage Limits	~10GB per user	Evidence storage <1GB (compressed files); audit logs <100MB
Compute Resources	Limited CPU/RAM	Async FastAPI prevents blocking; module execution parallelizable
No Outbound Email	Cannot send reports directly	Download-based workflow (PDF/JSON downloads)
Package Installation	pip install allowed with restrictions	All dependencies available via PyPI; provide requirements.txt

6.5 Deployment Checklist for NYU Sandbox

Pre-Deployment (Project 2 Setup): 1. Verify Python 3.10+ available (python --version) 2. Install dependencies: pip install -r requirements.txt 3. Configure OpenAI API key: export OPENAI_API_KEY=sk-... 4. Test ChatGPT connectivity: python test_openai_connection.py 5. Initialize SQLite database: python init_db.py 6. Load JSON schemas: python load_schemas.py

Runtime Requirements: - **Minimum:** 2 CPU cores, 4GB RAM (serial module execution) - **Recommended:** 4 CPU cores, 8GB RAM (parallel module execution) - **Storage:** 2GB (application + dependencies + audit logs)

API Usage Estimates: - **Per Audit:** ~280 ChatGPT API calls (28 criteria × ~10 calls each for analysis) - **Token Usage:** ~500K tokens per audit (GPT-4: ~\$10/audit at current pricing) - **Time:** ~15-20 minutes per complete audit (depends on evidence volume)

6.6 Technology Roadmap

MVP (Project 2 - Spring 2026): - Core module implementation (5 modules) - ChatGPT integration (criterion-level evaluation) - Basic web interface (Streamlit or FastAPI + HTML) - SQLite storage - PDF report generation

Enhanced (Project 3 - Fall 2026 if continued): - Parallel module execution (async optimization) - Advanced synthesis (ML-based divergence detection) - PostgreSQL migration (production-ready storage) - User authentication (multi-auditor support) - Audit history & analytics dashboard

Future (Post-Capstone): - Fine-tuned LLM for audit domain - Integration with AIS vendors (SAP, Oracle APIs) - Real-time monitoring mode (continuous auditing) - Multi-language support (GDPR compliance for EU audits)

7. Interface Design Principles for Projects 2 & 3

7.1 Modularity Guidelines

Each module MUST: 1. Implement BaseModule interface (Python ABC - Abstract Base Class) 2. Accept standardized input JSON (per schema 4.1) 3. Return standardized output JSON (per schema 4.1) 4. Have zero side effects (stateless execution) 5. Log all ChatGPT API calls (for reproducibility)

Example Module Structure:

```
from abc import ABC, abstractmethod

class BaseModule(ABC):
    @abstractmethod
    def evaluate(self, module_input: dict) -> dict:
        """
        Evaluate AI system for this module's criteria.
        Args:
            module_input: Validated JSON per schema 4.1
        Returns:
            module_output: JSON per schema 4.1
        """
        pass

    @abstractmethod
    def get_criteria_ids(self) -> list[str]:
        """Return list of criterion IDs this module evaluates."""
        pass

# Implementation example
class GovernanceModule(BaseModule):
    def evaluate(self, module_input: dict) -> dict:
```

```

# Load criteria (G1.1 - G1.7)
# For each criterion, call ChatGPT
# Aggregate scores
return module_output

def get_criteria_ids(self) -> list[str]:
    return ["G1.1", "G1.2", "G1.3", "G1.4", "G1.5", "G1.6", "G1.7"]

```

7.2 Testing Strategy

Unit Tests (Per Module): - Test criterion evaluation with sample evidence - Verify score ranges (0.0-1.0) - Validate output schema compliance - Mock ChatGPT responses (deterministic testing)

Integration Tests (Cross-Module): - Test orchestrator → module communication - Verify synthesis layer interdependency triggers - Test HITL escalation logic - End-to-end audit workflow

Performance Tests: - Measure time per criterion evaluation - Identify bottlenecks (ChatGPT latency) - Test parallel execution gains

8. Risks & Mitigation

Risk	Impact	Likelihood	Mitigation
OpenAI API unavailable in NYU Sandbox	HIGH	MEDIUM	Verify sandbox “API Access” mode enabled; provide Azure OpenAI fallback option
ChatGPT hallucination (incorrect scores)	HIGH	MEDIUM	Low temperature (0.2); structured output (JSON mode); human review for Material Weaknesses
Rate limiting (429 errors)	MEDIUM	HIGH	Exponential backoff; batch processing; cache identical evidence
Evidence quality poor (incomplete docs)	HIGH	HIGH	Validation checks; flag missing evidence; HITL escalation for ambiguous cases
Module execution time too long	MEDIUM	MEDIUM	Async parallel execution; optimize ChatGPT prompts; show progress bar
SQLite performance degradation	LOW	LOW	Adequate for MVP (<100 audits); migration path to PostgreSQL documented

9. Success Metrics (Architecture Quality)

Clear Module Boundaries: Each module has well-defined scope (governance, fairness, security, explainability, accuracy) - No overlap in criterion assignments - Dependencies explicitly documented (interdependency triggers)

I/O Contracts Specified: JSON schemas defined for all interfaces (sections 4.1, 4.2, 4.3) - Input validation via jsonschema library - Output validation automated in tests

NYU Sandbox Feasibility: All technologies available via pip - No external database services required - Deployment checklist provided (section 6.5)

Project 2 Implementation Support: BaseModule interface defined (section 7.1) - Testing strategy outlined (section 7.2) - Technology roadmap provides phasing guidance (section 6.6)

10. Conclusion

This architecture provides a solid foundation for implementing the AI Trust & Safety Auditing Agent in Projects 2 and 3. The “Council of Experts” pattern ensures modularity and specialization, while the three-stage synthesis process (independent scoring → peer review → divergence handling) guarantees comprehensive and reliable audits. The selected technology stack—Python, FastAPI, ChatGPT, and SQLite—balances sophistication with practical constraints of the NYU AI Sandbox environment.

Key Strengths: 1. **Modularity:** Clear boundaries enable parallel development by multiple team members 2. **Transparency:** All decisions (scores, triggers, escalations) are auditable 3. **Scalability:** Async architecture supports future enhancements 4. **Compliance:** GDPR/CCPA privacy integration from the ground up

Recommended Next Steps for Projects 2 & 3: 1. Implement BaseModule interface and 5 module classes 2. Develop Orchestrator for module coordination 3. Build Synthesis Layer with interdependency trigger engine 4. Create web interface (Streamlit recommended for MVP simplicity) 5. Conduct end-to-end testing with sample AI systems